

The Unilateral A2 Note

An Identity-Targeted Private Cheque Whose Recipient Alone Can Produce a Plaintext,
Bilateral Receipt

Perry Kundert

2026-06-03



A direct EOA transfer *deduces* identity from the two accounts in play – the account is primary, the Identity point M is derived. A Note is the opposite object: it *names* the identities and treats accounts as interchangeable envelopes that need only *match* the expected M . This document specifies, and reports a working implementation of, the **identity-targeted A2 Note**: a private-issuer cheque addressed to the recipient’s Identity M_{rec} rather than to one of the recipient’s accounts. The issuer encrypts its *own* registered Identity M_I under the recipient’s Identity *point*, so the one secret every Fountain account of the recipient shares – the identity scalar m_{rec} – opens it, and the issuer needs only M_{rec} , never an account. At deposit the recipient proves eligibility to the EVM with an Okamoto sigma that reveals no Identity; and from the same m_{rec} the recipient *alone* decrypts M_I and produces a plaintext receipt naming both parties and the amount. A colluding issuer cannot mint a spendable-but-unnameable note, because the decrypted issuer Identity must be a member of the registry-Identity tree. Every step is implemented and tested (`alberta_buck.wallet.unilateral_a2`, `IdentityRegistry.verifyDepositCoupling`) (Notes, Decryptability)

Contents

1	The Inversion: Address the Identity, Not the Account	3
2	Parties, Keys, and the Identity Tree	4
3	The Mint, Step by Step (Issuer)	5
3.1	Step M1 – recover the issuer’s own Identity	5
3.2	Step M2 – encrypt the issuer Identity under the recipient Identity point	5
3.3	Step M3 – the anti-framing binding (so <i>eIss</i> names the true minter)	5
3.4	Step M4 – commit the note	5
3.5	What the mint publishes, holds, and transmits	5
4	The A2 Note Payload (What Travels)	6
5	Delivery and Recipient Verification (Off Chain, Before Accepting)	7
6	The Deposit, Step by Step (Depositor)	8
6.1	The eligibility relation	8
6.2	The membership half (the SNARK piece)	8
6.3	Completing the spend	8
6.4	What the deposit publishes – and why Mallory is blind	8
7	The Unilateral Receipt (Recipient Alone)	10
7.1	Construction	10
7.2	Third-party verification (no secret)	10
7.3	Why it is unilateral, and why collusion fails	10
8	Security Properties (and the Tests that Witness Them)	11
8.1	Test ledger	11
9	Implementation Status and Remaining Work	12
9.1	Cross-references	13

STATUS (2026-06-10): historical – superseded. The A2 corner documented here is shipped and now lives in *alberta-buck-notes-flow* ("A2 Identity-Targeted Spend", "A2 in Code" – an executable transcript of this memo's flow) with formal statements in *alberta-buck-proofs* (Theorems 7-8, 12). The implementation-status section below reflects June 2026 and is no longer maintained.

1 The Inversion: Address the Identity, Not the Account

In the standard A2 construction (*alberta-buck-notes*) the issuer reads *one specific* registered account of the recipient and re-randomises *that* account's credential; the depositor must then spend from that same account. The identity-targeted construction drops the account from the addressing entirely. The issuer knows only the recipient's Identity point $M_{\text{rec}} = m_{\text{rec}}G$ – learned out of band, exactly as a payer learns "who" they are paying – and the note is redeemable by *any* account the recipient controls that is bound to m_{rec} .

This is not merely more convenient; it is *lighter to make collusion-resistant* and it yields a receipt the recipient can produce *alone*. The whole design turns on one substitution: the issuer encrypts under the recipient's Identity *point* M_{rec} , whose decryption secret m_{rec} is shared by every Fountain account of the recipient, instead of under one account's key pk_{rec} .

What was built. This document is matched by a working reference:

- `alberta_buck/wallet/unilateral_a2.py` – the mint, the deposit-coupling sigma, and the unilateral receipt (the registry-Identity Merkle tree is the canonical `alberta_buck.registry.tree.IdentityM` with `IdentityTree` here a thin point-centric facade over it).
- `IdentityRegistry.verifyDepositCoupling` – the on-chain, EIP-196 verification of the deposit-eligibility sigma.
- `Notes.spendCoupledA2` – the deposit entrypoint that co-verifies the coupling sigma and the P_I -bound G1-tie membership, then pays out.
- `circuits/identity_membership_g1tie.circom + IdentityMembershipG1TieVerifier(Adapter)` – the membership SNARK and the on-chain limb-binding of P_I .
- `alberta_buck/test/test_unilateral_a2.py` (11) plus `test/UnilateralA2.t.sol` (7), `test/IdentityMem` (5), and `test/NotesCoupledA2.t.sol` (8) – the end-to-end flow, the Solidity $\leftrightarrow$ Python cross-artifact parity, and the binding. All pass.

2 Parties, Keys, and the Identity Tree

Definition 1. *Notation.* G is the BN254 $\mathbb{G}_1$ generator; H a second nothing-up-my-sleeve generator ($= \text{issuer_reenc.H_POINT}$). An Identity is a KYC-bound scalar $m = H(id)$ and its point $M = mG$. A registered account is $(addr, pk = skG, E_{addr} = (R, C) = (rG, M + rpk))$: an ElGamal encryption of its Identity M under its own key. The Identity Fountain lets one m back unlimited unlinkable accounts $(sk_i, pk_i, E_{addr,i})$, all encrypting the same $M = mG$.

Two principals:

- **Issuer I :** identity (m_I, M_I) , with a registered account $(pk_I, E_I^{\text{reg}} = (R^{\text{reg}}, C^{\text{reg}}))$. Holds sk_I . The issuer recovers its own $M_I = C^{\text{reg}} - sk_I R^{\text{reg}}$.
- **Recipient D :** identity $(m_{\text{rec}}, M_{\text{rec}})$, with one or more registered accounts. The depositing account is $(pk_{\text{dep}}, E_{\text{dep}} = (R_d, C_d))$; D holds sk_{dep} and m_{rec} .

The identity provider and its tree. The registry maintains a Poseidon Merkle tree whose leaves are registered Identity *points*, $\ell(M) = \text{Poseidon}(M.x, M.y)$ (**identity_leaf**). Its root rt_{id} is published on chain; the full tree lives off chain (the Alberta Buck wallet's Holochain layer – no size limit). Membership of a point M in this tree is the statement " M is a genuine, KYC-registered Identity," and it is the anchor each party uses to certify the *other's* decrypted Identity.

3 The Mint, Step by Step (Issuer)

The issuer holds (sk_I, E_I^{reg}) and knows M_{rec} . It performs four operations.

3.1 Step M1 – recover the issuer’s own Identity

$$M_I = C^{\text{reg}} - sk_I R^{\text{reg}}.$$

Required: the issuer’s secret sk_I and its registered credential E_I^{reg} . *From the registry:* nothing new (the issuer reads its own record).

3.2 Step M2 – encrypt the issuer Identity under the recipient Identity point

With fresh randomness r' ,

$$eIss = (R_e, C_e) = (r' G, M_I + r' M_{\text{rec}}).$$

This is an ElGamal encryption of M_I under M_{rec} *treated as a public key*. Its decryption secret is m_{rec} , since $M_{\text{rec}} = m_{\text{rec}} G$: for any holder of m_{rec} ,

$$C_e - m_{\text{rec}} R_e = M_I + r' M_{\text{rec}} - m_{\text{rec}} (r' G) = M_I + r' (M_{\text{rec}} - m_{\text{rec}} G) = M_I.$$

Required from the issuer: M_I, r' . *Required of the recipient:* only M_{rec} – **not** an account, a key, or which account will deposit.

3.3 Step M3 – the anti-framing binding (so $eIss$ names the true minter)

The issuer proves, with the shipped recipient-blinded re-encryption sigma (`issuer_reenc`, here with the "recipient key" slot fed the Identity point M_{rec}), that $eIss$ re-encrypts the issuer’s *own registered* M_I – without revealing M_I or M_{rec} . This forecloses *framing*: the issuer cannot key $eIss$ over a victim’s Identity, because the proof is taken against $E_I^{\text{reg}} = E_{\text{addr}}[\text{msg.sender}]$. Verified on chain by `IdentityRegistry.verifyIssuerReenc`.

3.4 Step M4 – commit the note

$$idHash = \text{Poseidon}(R_e.x, R_e.y, C_e.x, C_e.y), \quad cm = \text{Poseidon}(A2, v, \rho, idHash, predicate).$$

The target Identity is *not* committed separately – it is implicit in $eIss$ ’s key, since only m_{rec} decrypts $eIss$ to a registered point (Section *Security*).

3.5 What the mint publishes, holds, and transmits

Datum	On chain (mint tx)	Issuer keeps	Sent to recipient
cm (opaque Poseidon commitment)	yes (calldata)	–	(implied by opening)
$eIss = (R_e, C_e)$	yes (leaf-tie public output)	r'	yes
anti-framing binding (<code>verifyIssuerReenc</code>)	yes	–	yes (for the receipt)
opening ($A2, v, \rho, idHash, predicate$)	no	–	yes (off chain)
$M_I, M_{\text{rec}}, sk_I, r'$	never	yes	–

To Mallory the mint is: `msg.sender` (the issuer account) escrowed v BUCK and folded an opaque cm , publishing a ciphertext $eIss$ and a zero-knowledge binding. Neither M_I (issuer Identity) nor M_{rec} (recipient Identity) is revealed.

4 The A2 Note Payload (What Travels)

The note is the opening tuple plus the issuer payload, delivered off chain (message, QR, or wallet-to-wallet):

```
note = (  
    flavor = A2,  v,  rho,  idHash,  predicate,      # the commitment opening  
    eIss    = (R_e, C_e),                          # issuer M under recipient identity point  
    binding,                                     # verifyIssuerReenc anti-framing proof  
    mint_tx_ref,                                # where cm was folded (anchors the receipt)  
)
```

It carries *no* account address of either party and *no* plaintext Identity. Anyone intercepting it learns nothing: *eIss* is semantically secure absent m_{rec} , and the binding is zero-knowledge.

5 Delivery and Recipient Verification (Off Chain, Before Accepting)

On receipt, D verifies the note *before* treating it as money:

1. **Decrypt the issuer.** $M_I = C_e - m_{\text{rec}}R_e$ (elgamal_decrypt under m_{rec}).
2. **Membership.** Check $\ell(M_I) \in \text{rt}_{\text{id}} - M_I$ is a registered Identity (`IdentityTree.contains`).
3. **Binding.** Check `verifyIssuerReenc` accepts the binding against the named issuer account's registry record – $eIss$ really re-encrypts *that account's* registered Identity.

If any check fails the recipient *rejects* the note (it is un-nameable, so its value could never be accounted for). An honest recipient thus only ever accepts notes it can later receipt – the cryptographic form of the recipient-burden posture. (Tested: `test_delivery_recipient_decrypts_issuer`.)

6 The Deposit, Step by Step (Depositor)

D chooses *any* registered account bound to m_{rec} as the payout vehicle and proves eligibility to the EVM while revealing no Identity.

6.1 The eligibility relation

The depositor proves knowledge of $(m_{\text{rec}}, sk_{\text{dep}}, b)$ such that, with $eIss = (R_e, C_e)$, the chosen account's registered $E_{\text{dep}} = (R_d, C_d)$, its on-chain key pk_{dep} , and a published commitment $P_I = M_I + bH$:

- (E4) $pk_{\text{dep}} = sk_{\text{dep}} G$ — the real account key;
- (E2) $C_d = m_{\text{rec}} G + sk_{\text{dep}} R_d$ — the account is bound to $M_{\text{rec}} = m_{\text{rec}} G$;
- (E3) $C_e - P_I = m_{\text{rec}} R_e - bH$ — $eIss$ decrypts under m_{rec} to $P_I - bH$.

(E4) pins sk_{dep} to the registered key; (E2) then pins $m_{\text{rec}} G$ to the account's Identity; (E3) ties that *same* m_{rec} to $eIss$'s decryption, with the recovered issuer Identity M_I *committed and blinded* in P_I . This is a five-point Okamoto sigma over EIP-196 (`deposit_couple_prove` / `deposit_couple_verify`, on chain `verifyDepositCoupling`) – the same family as `verifyApprove` / `issuer_reenc`, and equally cheap.

6.2 The membership half (the SNARK piece)

P_I commits the decrypted issuer Identity. The depositor additionally proves, in zero knowledge, that P_I 's point is a member of the registry-Identity tree: $\exists b : P_I - bH = M_I \wedge \ell(M_I) \in \text{rt}_{\text{id}}$. The membership is a *native* Poseidon-Merkle proof; the tie to the EC commitment P_I is the one non-native scalar relation, and the only remaining circuit work (Section *Implementation Status*). Its effect: a note whose $eIss$ is keyed to anything but M_{rec} decrypts to a non-member and the spend *reverts*.

6.3 Completing the spend

The deposit calls (schematically) `Notes.spend(proof, root, nullifier, face, recipient, eIss, depositCouplingProof, membershipProof)`, where the contract: verifies the note's Merkle inclusion and nullifier (unspent), calls `verifyDepositCoupling(depositor, eIss, dc)` (E2/E3/E4), checks the membership proof against rt_{id} , records the nullifier, and pays `face` BUCK out of the pool to the depositor's chosen account via the Carrying path. The nullifier is $\text{Poseidon}(\rho, idHash, A\text{-tag})$ (`nullifier_a`), unlinkable to *cm*.

6.4 What the deposit publishes – and why Mallory is blind

Datum	Revealed at deposit	Hidden
depositor account (<code>msg.sender</code>)	yes (it receives the BUCK)	its Identity M_{rec} (DDH in E_{dep})
$pk_{\text{dep}}, E_{\text{dep}}$	yes (already in registry)	$sk_{\text{dep}}, m_{\text{rec}}$
$eIss, P_I$	yes	M_I (blinded in P_I), M_{rec}
nullifier, face	yes	which mint this redeems
issuer account / tx	never (not an input at deposit)	the entire issuer side

The deposit names *no* issuer account, tx, or Identity. $eIss$ and P_I are the only Identity-derived values, and both are blinded: $eIss$ is semantically secure, $P_I = M_I + bH$ is perfectly hiding. So Mallory cannot link the depositor's account to the issuer's mint, nor to other accounts of the same recipient. (Tested: `test_deposit_proof_hides_identities`, `test_two_mints_unlinkable`; soundness:

`test_deposit_coupling_{tamper,wrong_account,wrong_chainid}_rejected` and the seven Solidity parity tests.)

7 The Unilateral Receipt (Recipient Alone)

The recipient holds the *one* secret m_{rec} that names *both* parties, and needs no cooperation from the issuer at spend time.

7.1 Construction

$$\begin{aligned} M_I &= C_e - m_{\text{rec}} R_e && \text{(the issuer Identity);} \\ M_{\text{rec}} &= m_{\text{rec}} G && \text{(the recipient's own).} \end{aligned}$$

The recipient attaches a *verifiable decryption* (`verifiable_decrypt`, a Chaum-Pedersen DLEQ) proving *eIss* decrypts under M_{rec} to M_I – revealing both points but not m_{rec} . The receipt is

$$\mathcal{R} = (M_I, M_{\text{rec}}, v, eIss, \text{vd}, \text{binding}, \text{issuer addr}, \text{chainid}).$$

7.2 Third-party verification (no secret)

A court, auditor, or counterparty checks four links (`verify_receipt`):

Link	Check	Establishes
1	<code>verifyIssuerReenc</code> on the binding vs. the issuer account's record	<i>eIss</i> re-encrypts the issuer's <i>registered</i> M_I (no framing)
2	<code>verifiable_decrypt_verify</code> (<i>eIss</i> , M_{rec} , M_I)	<i>eIss</i> decrypts under M_{rec} to M_I
3	$\ell(M_I) \in \text{rt}_{\text{id}}$	the issuer Identity is registered – <i>the coupling</i>
4	$\ell(M_{\text{rec}}) \in \text{rt}_{\text{id}}$	the recipient Identity is registered

Link 3 is the load-bearing one. Composing 1 and 2: $M_I = C_e - r' M_{\text{rec}}$ (link 2, since $R_e = r' G$) while the binding gives $C_e = M_I^{\text{reg}} + r' Q$ for the issuer's registered Identity M_I^{reg} and the issuer-chosen point Q . Hence $M_I = M_I^{\text{reg}} + r'(Q - M_{\text{rec}})$. If $Q \neq M_{\text{rec}}$ the recovered M_I is a pseudorandom offset of a registered point and is *not* itself in the tree; so link 3 *forces* $Q = M_{\text{rec}}$ and thus $M_I = M_I^{\text{reg}}$. The receipt is therefore VALID exactly when it names the true issuer. (Tested: `test_unilateral_receipt_names_both`.)

7.3 Why it is unilateral, and why collusion fails

Completeness needs no issuer participation: m_{rec} yields both identities. Collusion-resistance is the contrapositive of link 3: a colluding issuer who keys *eIss* to a throwaway point (so the recipient "cannot" decrypt to a registered Identity) produces a note whose decrypted M_I is a non-member – so the *receipt is INVALID* and the *spend membership reverts*. The pair cannot obtain a spendable-but-unnameable note. (Tested: `test_collusion_bogus_eiss_unnameable` – the bogus note's binding still verifies, yet the decrypted point is provably absent from the tree and `verify_receipt` returns INVALID at link 3.)

8 Security Properties (and the Tests that Witness Them)

Theorem 2. (Completeness.) *For an honest mint to M_{rec} , any account of the recipient bound to m_{rec} satisfies E2/E3/E4 and the membership; and the recipient produces a VALID receipt naming (M_I, M_{rec}, v) .*

Theorem 3. (Collusion-resistance.) *Under the non-grindable-identities assumption ($M = H(id)G$, H a random oracle), no issuer/recipient coalition produces a spendable note whose decrypted issuer Identity is absent from the tree: spendability requires $eIss$ to decrypt, under the depositor's authenticated m_{rec} , to a tree member, which is nameable.*

Theorem 4. (Non-frameability.) *The mint binding forces $eIss$ over the minter's own registered M_I ; the only reachable outcomes are "names the true minter" (VALID) or "names no one" (INVALID). A victim cannot be substituted.*

Theorem 5. (Privacy.) *The deposit sigma is honest-verifier zero-knowledge and P_I is perfectly hiding; $eIss$ is IND-CPA. No third party learns m_{rec} , M_{rec} , or M_I , and the deposit names no issuer account or transaction – so mint and spend stay unlinkable.*

The cross-artifact tests pin the Solidity verifier to the Python reference byte-for-byte (the established `alberta_buck.wallet` $\rightarrow$ `vm.readFile` pattern): `scripts/gen_unilateral_a2_vectors.py` emits `test/vectors/unilateral_a2.json`; `test/UnilateralA2.t.sol` reads it and asserts `verifyDepositCoupling` accepts the canonical proof and rejects every tamper.

8.1 Test ledger

Suite	Count	What it demonstrates
<code>alberta_buck/test/test_unilateral_a2.py</code>	11	mint binding; delivery decrypt; deposit coupling; unilateral receipt names both; any-ac
<code>test/UnilateralA2.t.sol</code>	7	on-chain <code>verifyDepositCoupling</code> parity + soundness (tampered response / P_I / $eIss$,
<code>test/IdentityMembershipBinding.t.sol</code>	5	the P_I binding: the bound point verifies; a mismatched P_{Ix} / P_{Iy} / root is rejected; b
<code>test/NotesCoupledA2.t.sol</code>	8	end-to-end <code>spendCoupledA2</code> : completeness + payout + <code>SpentCoupledA2</code> ; bad coupling /

Status: all 31 pass (make `nix-match-NotesCoupledA2`, make `nix-match-IdentityMembershipBinding`, make `nix-match-UnilateralA2`, and `nix develop --command python -m pytest alberta_buck/test/test_uni` with no regression in the adjacent identity / A2 suites (make `nix-test`: 425 forge passed; the 8 failures are the pre-existing uniswap-artifact and manual-fixture env tests).

9 Implementation Status and Remaining Work

Shipped and tested:

- The mint, payload, off-chain delivery verification, deposit-coupling sigma, and unilateral receipt – the full cryptographic flow – in `alberta_buck/wallet/unilateral_a2.py`.
- The on-chain deposit-eligibility verifier `IdentityRegistry.verifyDepositCoupling` (EIP-196), with the `DepositCouplingProof` struct and a Fiat-Shamir transcript identical to the wallet's.
- The registry-Identity Poseidon Merkle tree (`IdentityTree`) and the receipt verifier (`verify_receipt`), with membership demonstrated in the reference.

Built and on-chain verified (Phase C, 2026-06-06):

- The G1-tie circuit (`circuits/identity_membership_g1tie.circom`) – 39K constraints, proving $P_I = M + bH$ in the G1 group over BN254 via circom-lib's `EllipticCurveAddOptimised`. Compiled with circom 2.2.3, proved with snarkjs 0.7.5, Groth16 verifier exported to Solidity (`src/IdentityMembershipG1TieVerifier.sol`).
- On-chain verification: all 3 Forge tests pass (valid proof accepted, tampered root rejected, tampered P_I rejected) at $\sim 350K$ gas.
- EIP-197 G2 encoding bug discovered and worked around: snarkjs stores G2 points as `[x_re, x_im, y_re, y_im]` but EIP-197 expects `[x_im, x_re, y_im, y_re]`. Post-processor `fix_verifier_g2.py` applies the swap automatically (see `alberta-buck-verifier-bug.org`).
- The incremental registry-Identity accumulator is deployed on chain (`IdentityRegistry.identityRoot`), updated on each `register()` / `bindContract()` call via on-chain Poseidon hashing.
- The identity membership verifier is wired into `Notes.sol` spend paths (`IIdentityMembershipVerifier.verifyIdentityMembership`), gated by governance.

The deposit is now bound and routed (2026-06-06):

- `Notes.spendCoupledA2` is the identity-M-bound A2 deposit entrypoint: it calls `verifyDepositCoupling(msg.eIss, dc)` and then `_verifyIdentityMembership(membershipProof, dc.P_I.X, dc.P_I.Y)`, **reading `dc.P_I` once and feeding it to both**. Emits `SpentCoupledA2(nullifier, face, recipient, P_I.X, P_I.Y)`.
- **The binding** – the step the earlier draft called "remaining": the membership verifier no longer trusts the prover's public inputs. `IIdentityMembershipVerifier.verifyMembership(proof, identityRoot, px, py)` takes the committed point, and `IdentityMembershipG1TieVerifierAdapter` derives the circuit's nine public signals (the 64-bit $\mathbb{F}_q$ limbs of P_I) on chain from `(px, py)`, so the `membershipProof` bytes carry only the Groth16 triple (a, b, c) . A Groth16 accept therefore proves membership of *exactly* the point the coupling sigma decrypted `eIss` to – the two halves cannot be decoupled.
- Tests: `test/IdentityMembershipBinding.t.sol` (5 – the bound point verifies; a mismatched P_I or root is rejected) and `test/NotesCoupledA2.t.sol` (8 – the full deposit against the canonical coupling vector + the stub membership: completeness, bad coupling / tampered P_I / unregistered depositor / rejected membership / double-spend all revert). `make nix-test` stays green (no regressions).

Remaining for full Identity-M Notes:

- The B1 depositor-binding sigma (`verifyDepositorBinding`) similarly needs its membership counterpart wired at spend time – the one symmetric increment still open (lower priority: a B1 issuer is public and named at mint, so the receipt does not depend on it).
- Production deployment requires a proper Powers of Tau ceremony (the current ptaus use dev-only entropy).

9.1 Cross-references

- *alberta-buck-notes-decryptability* – the gap this closes (the A2 recipient-key coupling) and the "identity-targeted alternative" subsection that this document realises.
- *alberta-buck-identity* – the Identity Fountain, registration, and the Chaum-Pedersen / ElGamal primitives reused here.
- Conventions: a negative signed BUCK balance is *used credit*, not "debt"; build/test via `nix develop --command`.